Bloque 2: Entrenamiento, validación y evaluación de redes neuronales

1 Introducción

En este segundo bloque entrenaremos una red neuronal (*modelo*) para detectar RoboMasters en las imágenes que anotamos en el Bloque 1. Usaremos **PyTorch** junto con *torchvision*, que proporciona modelos de detección de objetos ya implementados.

Un detector recibe una imagen y devuelve, para cada objeto que encuentra, una caja (*dónde* está el objeto) y una etiqueta con una confianza (*qué* es ese objeto). En lugar de aprender todo esto desde cero, partiremos de modelos **preentrenados** sobre grandes bases de datos y solo readaptaremos su parte final a nuestras clases. Esta técnica, conocida como **transfer learning**, permite obtener buenos resultados con muchas menos imágenes y en mucho menos tiempo.

A lo largo del bloque aprenderá a preparar los datos para la red, elegir y configurar un modelo, entrenarlo, validarlo mientras entrena y evaluarlo sobre secuencias que nunca ha visto.

2 Punto de partida

Seguiremos trabajando en el JupyterHub del laboratorio (`https://k8ssr.labdoc.ssr.upm.es:30000`, perfil *PIDS*). Si completó correctamente el Bloque 1, su perfil debería tener esta estructura:

```
data_train/          <- SUS secuencias, anotadas y depuradas
    video1/
        images/
        labels/
    video2/
        ...
data_val/            <- secuencias de validación (proporcionadas)
data_test/           <- secuencias de test (proporcionadas)
models/              <- se creará sola al entrenar
week1/
week2/
    train.ipynb
    test.ipynb
```

Trabajaremos con los dos notebooks de *week2*: train.ipynb para entrenar y validar, y test.ipynb para evaluar.

3 Entrenamiento: *train.ipynb*

Abra *week2/train.ipynb* y ejecute las primeras celdas para importar las librerías. La sección *Utils functions* contiene funciones auxiliares que se usan a lo largo de todo el notebook; no es imprescindible entenderlas todas, pero sí es recomendable leer sus *docstrings*. Además, la función *torchvision_model* contiene los modelos disponibles en el notebook.

3.1 Generador de datos

Cuando la base de datos tiene miles de imágenes, es inviable cargarlas todas en memoria y procesarlas a la vez. En su lugar se usan **generadores de datos**, que van entregando a la red subconjuntos de imágenes anotadas llamados *batches*. Cuando se han procesado todas las imágenes de la base de datos, se dice que la red ha entrenado durante un *epoch*. Conseguir un buen aprendizaje suele requerir decenas o centenares de epochs.

En PyTorch esto se reparte entre dos piezas que conviene no confundir:

- El *Dataset* (clase *CustomDataset*) carga **una** muestra: abre la imagen, lee sus anotaciones y aplica el preprocesado.
- El *DataLoader* agrupa varias muestras en un **batch** y se lo pasa a la red.

Estudie el método *__getitem__* y compruebe el formato exacto de lo que devuelve, porque debe coincidir con lo que espera el modelo: la imagen como tensor y un diccionario con *boxes* (coordenadas [x_min, y_min, x_max, y_max]) y *labels*. Preste atención también a *get_transform*: explique qué preprocesado se aplica a las imágenes y por qué.

3.2 Modelos y transfer learning

La función *torchvision_model* contiene un diccionario con los modelos disponibles: *faster_rcnn_v1*, *faster_rcnn_v2*, *faster_rcnn_v3*, *retinanet_v1*, *retinanet_v2*, *ssd_v1* y *ssd_v2*. Use estas claves en el parámetro *model_name* para elegir qué red entrenar.

> Esas claves son solo nombres cómodos: el nombre real de cada modelo es el que aparece en los valores del diccionario. En la memoria no queremos leer *faster_rcnn_v1*, sino *Faster R-CNN con un backbone ResNet50-FPN*.

No se espera que entienda al detalle el funcionamiento interno de estas arquitecturas. Basta con que, de cada modelo que llegue a comparar, sepa explicar en unas pocas frases en qué familia se encuadra, qué idea lo caracteriza y en qué se diferencia de los demás. Investigue también qué es un *backbone*, qué papel cumple dentro de un detector y por qué se habla de él por separado del resto de la red.

La misma función implementa el transfer learning. Con *pretrained = True*, el modelo se construye con los pesos aprendidos sobre COCO, una base de datos de propósito general con decenas de miles de imágenes; a continuación se **sustituye su cabeza de clasificación** (las capas finales de predicción) por una nueva, adaptada a nuestro problema. De ahí que *num_classes* sea siempre 2: la clase 0 es el fondo y la 1, el robot. El resto de la red conserva lo que ya sabía sobre bordes, texturas y formas, y solo tiene que ajustarlo a nuestro objeto.

3.3 Hiperparámetros

La sección *Parameters* reúne todo lo que puede configurar, agrupado en cuatro bloques:

- **Datos:** las rutas a *data_train* y *data_val*.
- **Modelo:** *model_name*, *num_classes*, *pretrained*, *use_gpu* y *resize_shape* (tamaño al que se redimensionan las imágenes, o *None* para dejarlas como están).

- **Entrenamiento:** *num_epochs*, *batch_size* y *num_workers*. Deje *num_workers* como está.
- **Optimizador:** el learning rate (*lr*) y los parámetros del SGD y del *learning rate scheduler* (*lr_factor*, *lr_patience*, *lr_threshold*, *lr_min*). También *max_grad_norm*, que limita la norma de los gradientes; solo hace falta para SSD.

La variable *SEED* fija todas las fuentes de aleatoriedad para que dos ejecuciones idénticas den exactamente el mismo resultado. No la cambie si quiere poder comparar experimentos entre sí.

El modelo se guarda en *models/{model_name}/*, en dos archivos con propósitos distintos:

- *model_last.pt*: se guarda **en cada epoch** y sirve para retomar un entrenamiento interrumpido por donde iba.
- *model_best.pt*: el modelo con la **menor pérdida de validación** hasta el momento. Es el que se usará para evaluar.

> **IMPORTANTE**: si vuelve a entrenar con el mismo *model_name*, el notebook encontrará el *model_last.pt* anterior y continuará aquel entrenamiento en lugar de empezar uno nuevo. Para arrancar de cero, cambie el nombre del modelo o borre (o renombre) su carpeta. Antes de lanzar cada prueba, asegúrese de que no está sobrescribiendo un modelo que quiera conservar.

3.4 Bucle de entrenamiento

Las primeras celdas de la sección *Train* crean el modelo, el optimizador, el *learning rate scheduler* (investigue qué es y para qué sirve) y los generadores de datos de entrenamiento y validación. La celda siguiente contiene el bucle de entrenamiento. En cada epoch se hace lo siguiente:

1. Poner el modelo en modo de entrenamiento.
2. Extraer un batch del generador de entrenamiento y moverlo a la GPU (o a la CPU).
3. Predecir y obtener la función de coste. En modo de entrenamiento, los modelos de torchvision devuelven directamente las pérdidas, sin necesidad de calcularlas aparte.
4. Poner a cero los gradientes acumulados en el optimizador.
5. Aplicar *backpropagation* para calcular los gradientes.
6. Limitar la norma de los gradientes (solo para algunas redes).
7. Dejar que el optimizador actualice los pesos a partir de esos gradientes.
8. Repetir los pasos 2–7 hasta agotar la base de datos.
9. Recorrer el conjunto de validación calculando la pérdida **sin** actualizar los pesos.
10. Actualizar el learning rate scheduler y guardar los checkpoints.

Por último, las celdas finales dibujan las curvas de pérdidas de entrenamiento y validación y la evolución del learning rate. Sepa leerlas: qué información aporta cada una y qué significa su forma (creciente, decreciente, plana, ruidosa, separada entre entrenamiento y validación...). Las comentará al presentar sus experimentos.

IMPORTANTE: pruebe el notebook hasta tener algo que funcione y solo entonces lance el entrenamiento completo. Un entrenamiento puede llevar en torno a una hora, y si cierra el navegador o pierde la conexión se detendrá (no es grave: guardamos un checkpoint en cada epoch y podrá retomarlo).

Para evitarlo, puede lanzar el entrenamiento en segundo plano. Pulse Ctrl+Shift+L, abra un Terminal y ejecute:

```
papermill train.ipynb {nombre-del-experimento}.ipynb
```

Abra de vez en cuando el notebook generado para ver por qué epoch va. Puede cerrar el navegador, pero **no cierre la pestaña del Terminal**. Para detener el entrenamiento, pulse Ctrl+C y cierre el Terminal.

Ponga a cada notebook un nombre que describa la prueba que está haciendo: le servirá de registro de experimentos. Si quiere encadenar varias pruebas de una vez, consulte la documentación de papermill y, en particular, cómo parametrizar y ejecutar notebooks.

4 Evaluación: *test.ipynb*

Evaluar consiste en medir el rendimiento del modelo sobre un conjunto de datos que **no ha visto durante el entrenamiento**: las secuencias de *data_test*. El notebook *test.ipynb* es muy parecido al anterior y reutiliza sus mismas funciones auxiliares y su mismo generador de datos, de forma que el modelo vea las imágenes exactamente igual que cuando aprendió. Indique el modelo a evaluar en *model_name* (se cargará automáticamente su *model_best.pt*) y ejecute el notebook entero.

Análisis cuantitativo. La métrica estándar en detección de objetos es la *mean Average Precision* (mAP). Investigue cómo se calcula esta métrica y qué mide exactamente cada una de las variantes que aparecen en el notebook.

Análisis cualitativo. Las métricas resumen el rendimiento en un número, pero pueden esconder algunos errores concretos. Las celdas finales permiten dibujar las predicciones sobre las imágenes. Busque casos en los que el modelo acierte y, sobre todo, casos en los que falle, e intente explicar por qué. Preste atención al parámetro *score_threshold*: determine para qué sirve y qué ocurre al subirlo y al bajarlo.

5 Objetivos

Partiendo de la base de datos del Bloque 1, debe entrenar un detector de RoboMasters lo más robusto posible y documentar el proceso. Concretamente:

- **Entender el pipeline:** sea capaz de describir, a grandes rasgos, los módulos del sistema y cómo se conectan entre sí.
- **Conocer los modelos:** empiece por *faster_rcnn_v1*. Investigue qué arquitectura es en realidad, qué es un *backbone* y en qué consiste el transfer learning que estamos aplicando. Describa brevemente cada modelo que llegue a comparar. Opcionalmente puede implementar por su cuenta otros modelos y comprobar si funcionan mejor.

- **Entender los hiperparámetros:** sepa explicar qué controla cada uno. Céntrese en el **batch size**, el **learning rate** y el **número de epochs**. Del resto, basta con saber para qué sirven. Tiene pistas sobre el optimizador y el scheduler aquí y aquí.
- **Entender las métricas:** investigue cómo se calculan la IoU, la precisión, el recall, la AP y la mAP, y qué diferencia hay entre mAP, mAP@0.50 y mAP@0.75.
- **Experimentar**, siguiendo este orden:
  1. **Ajuste los hiperparámetros** con un único modelo. Batch size: ¿qué ocurre con valores muy bajos? ¿Y con valores muy altos? ¿Aparece algún error de ejecución? Learning rate: ¿qué ocurre con valores muy bajos? ¿Y muy altos? ¿Afecta al número de epochs necesarios? La gráfica del learning rate le dará pistas. Epochs: ¿hasta cuándo merece la pena entrenar? No malgaste tiempo entrenando hasta el infinito.
  2. **Compare varias arquitecturas** dejando fijos esos hiperparámetros, y quédese con la mejor. Si alguna red le da problemas, revise *max_grad_norm* y la función *clip_grad_norms*, y explique qué hacen y por qué son necesarias.
  3. **Añada los datos aumentados** del Bloque 1 al final, reentrenando solo el mejor modelo, para intentar un último empujón en la mAP.

  Este orden importa: entrenar todos los experimentos con datos aumentados multiplicaría el tiempo de cómputo sin aportar información nueva. Anote cada prueba que haga, incluidas las que salgan mal.
- **Analizar cualitativamente:** busque imágenes concretas en las que el modelo funcione bien y, sobre todo, imágenes en las que falle, y caracterice qué tipo de imágenes le hacen fallar (objetos lejanos, oclusiones, escenas oscuras, desenfoque...). Relacione esos fallos con lo que ya sabía de sus datos.
- **(Opcional)** Mida también los tiempos de inferencia, no solo la precisión: le permitirá comparar la velocidad de los modelos, algo que importará en el Bloque 3.

5.1 Entregable

La memoria es **incremental**: se entrega tres veces y, en cada entrega, se corrigen los bloques anteriores y se añade el nuevo. Siga utilizando la plantilla *PIDS-template.docx*. Para esta segunda entrega:

- **Contenido:** el Bloque 1 corregido y el Bloque 2 completo. Rellene también la tabla "Respuesta a la revisión anterior": su ausencia penaliza la nota.
- **Extensión máxima:** 20 páginas de cuerpo, contando el Bloque 1. No cuentan la portada, el índice, la respuesta a la revisión anterior, el reparto del trabajo, las referencias ni los anexos.
- **Fichero:** un único PDF llamado **PIDS_grupoX_entregaN.pdf**, donde X es el número de grupo y N el de la entrega. Por ejemplo, el grupo 3 entregaría *PIDS_grupo3_entrega2.pdf*.

Se valorará que las pruebas realizadas tengan sentido y estén fundamentadas, y que las conclusiones se apoyen en resultados medidos y no en intuiciones.